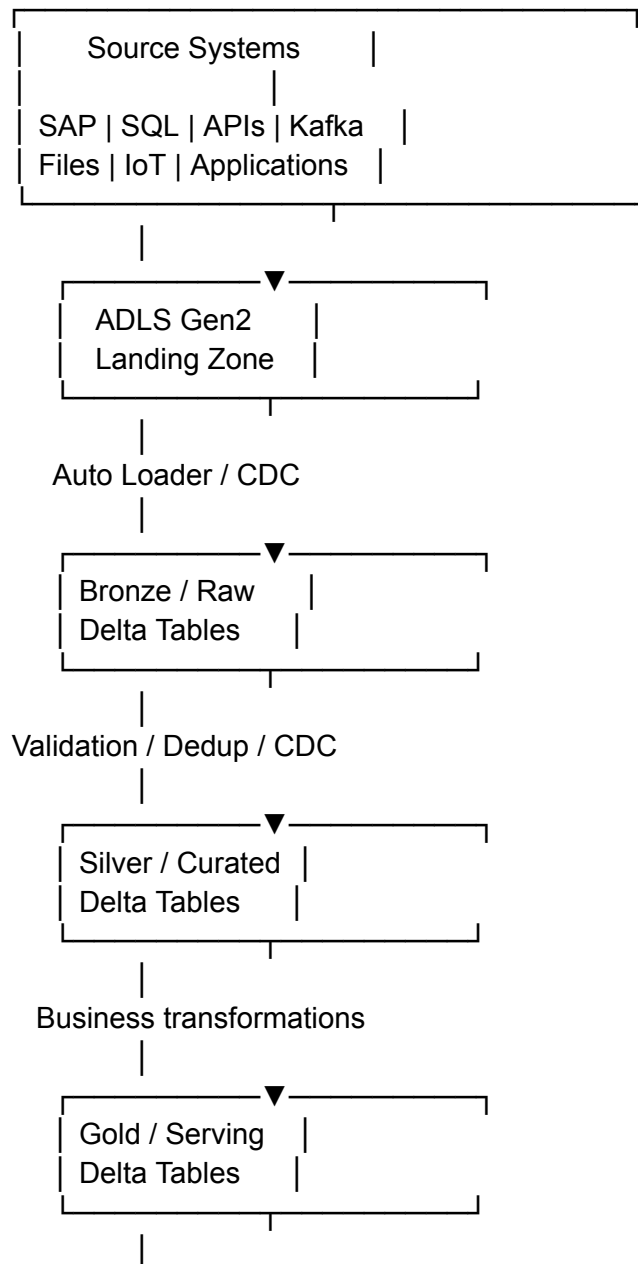
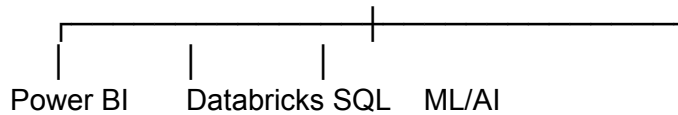


Databricks Interview Questions — Detailed Answers

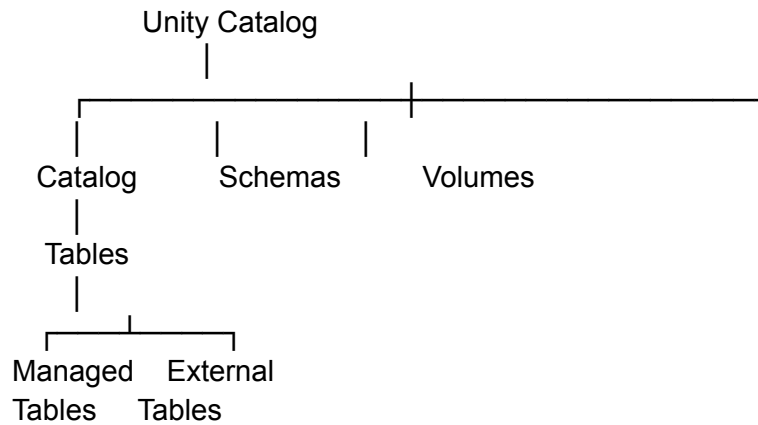
Overall reference architecture

Before going question-by-question, keep this production architecture in mind:

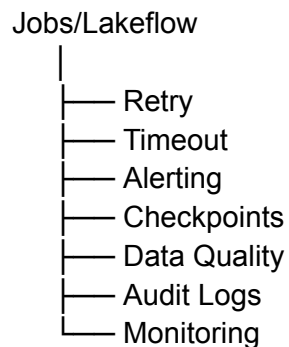




Governance:



Operationally:



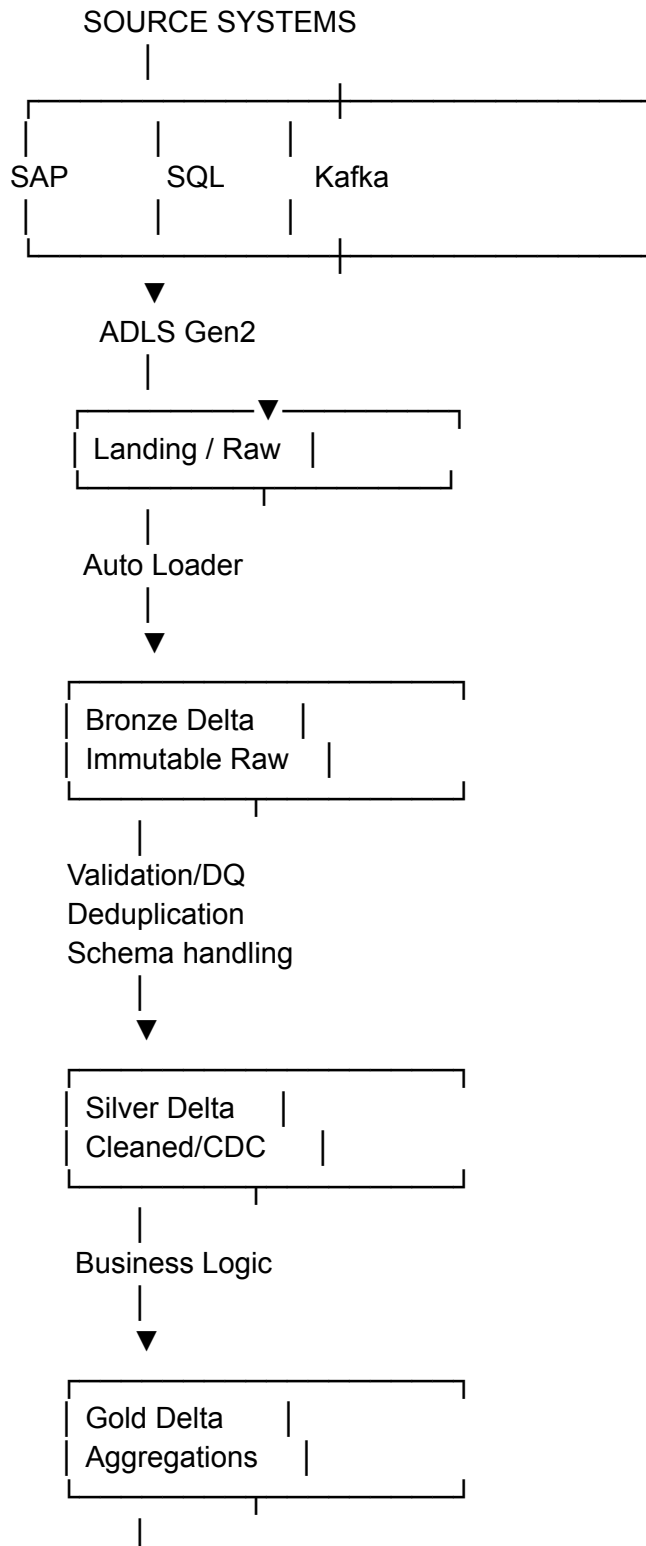
Databricks currently recommends Auto Loader for incremental cloud-file ingestion, and file-events-based discovery is recommended for most production workloads because it scales better than repeated directory listing.

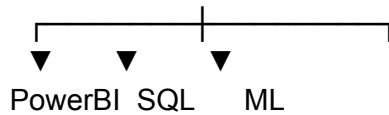
1. Design a production Databricks architecture for 5 TB/day

Answer

For 5 TB/day, I would design a **Lakehouse architecture using ADLS Gen2 + Databricks + Delta Lake + Unity Catalog.**

Architecture





Bronze

Store data as close to the source as possible.

For example:

/catalog/raw/sap/customer

/catalog/raw/sap/orders

/catalog /raw/iot/sensor

Don't perform heavy transformations here.

Silver

Perform:

- schema validation
- deduplication
- CDC processing
- data type standardization
- null handling
- reference-data joins
- business keys

Gold

Create:

- fact tables
- dimension tables
- aggregates
- reporting datasets
- ML features

How would you handle 5 TB/day?

I would focus on:

1. Incremental processing

Never scan all historical data every day.

Yesterday:

5 TB total

20 GB changed

Process:

20 GB

NOT

5 TB

2. Auto Loader

For file ingestion:

```
df = (  
    spark.readStream  
    .format("cloudFiles")  
    .option("cloudFiles.format", "json")  
    .option("cloudFiles.schemaLocation", schema_path)  
    .load(source_path)  
)
```

3. Delta Lake

Use Delta for:

- ACID transactions
- schema enforcement
- MERGE
- time travel
- CDC
- data skipping
- reliable streaming

4. Unity Catalog

Use:

```
prod_catalog  
|  
├── bronze  
├── silver  
└── gold
```

and grant access using groups rather than individual users.

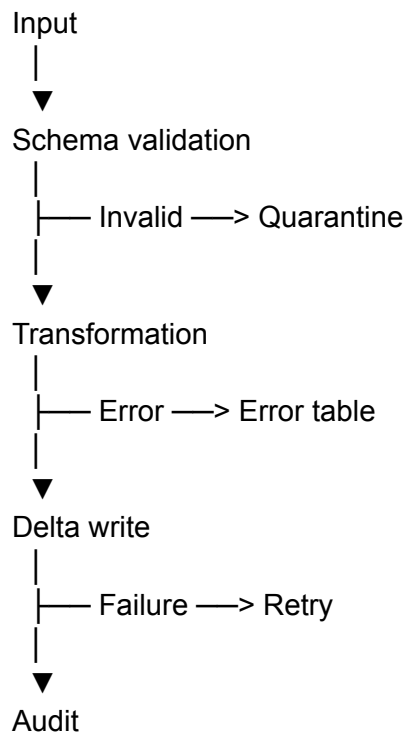
5. Compute

Use:

- Photon where appropriate
- autoscaling
- job clusters/serverless where suitable
- separate ingestion and transformation workloads
- workload-specific clusters

Error handling

Every pipeline should have:



Create an audit table:

```
pipeline_audit
-----
run_id
pipeline_name
source
start_time
end_time
```

status
records_read
records_written
records_rejected
error_message
batch_id

Interview answer

“For 5 TB/day, I would avoid full scans and design an incremental architecture using Auto Loader or source CDC, Bronze/Silver/Gold Delta tables, Unity Catalog for governance, Photon and autoscaling for compute, and checkpointing for streaming reliability. I would also implement data-quality quarantine, audit logging, retries and alerting.”

2. Why Auto Loader instead of

`spark.read`

on ADLS?

This is a very important interview question.

Simple answer

`spark.read` is fundamentally a batch operation.

For example:

```
df = spark.read.parquet("/data/orders")
```

Every execution must discover/read the files according to the batch query.

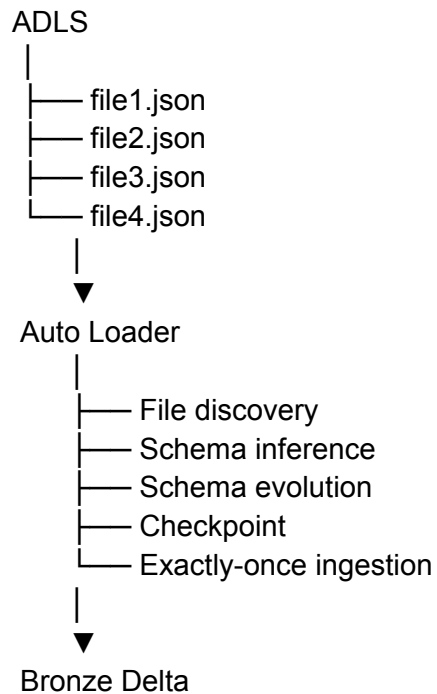
Auto Loader:

```
spark.readStream \  
  .format("cloudFiles")
```

is designed specifically for **incremental file ingestion**.

Auto Loader tracks which files have already been processed and can scale to extremely large numbers of files. Databricks documents support for billions of files and recommends Auto Loader for incremental ingestion.

Architecture



Auto Loader can use file events/notifications rather than repeatedly listing the entire directory. Current Databricks guidance recommends file events for most workloads.

Important distinction

Auto Loader provides **exactly-once file ingestion semantics**, but that does not mean your downstream business transformation automatically deduplicates duplicate business events.

For example:

Order 100
Order 100
Order 100

could still represent duplicate business events if the source generated them.

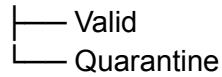
You need business-key deduplication.

Error handling

Use:

Bad records

Bronze

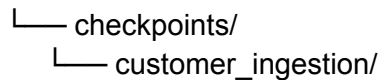


Schema evolution

Configure schema handling and rescue unexpected fields where appropriate.

Checkpoint

ADLS



Never casually delete the checkpoint.

Databricks specifically warns that changing the checkpoint location effectively starts a new stream.

3. MERGE processes 2 TB even though only 20 GB changed. How do you fix it?

This is a **very good senior-level question**.

Problem

You have:

Target = 2 TB

Today's changes = 20 GB

But:

MERGE source

INTO target

causes massive scanning.

The goal is:

2 TB



identify affected files/partitions



20 GB-ish candidate data



MERGE

Solution 1 — Incremental source

Don't send the entire source table.

Instead:

Source CDC



20 GB changed records



MERGE

Solution 2 — Filter the target

Suppose the table is organized by:

business_date

and today's CDC only affects:

2026-08-28

Don't scan every historical date.

Example:

MERGE INTO target t

```
USING source s
ON t.customer_id = s.customer_id
AND t.business_date = s.business_date
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

Solution 3 — Data skipping / clustering

Delta maintains per-file statistics such as min/max values, allowing irrelevant files to be skipped.

For modern Databricks environments, consider **liquid clustering** rather than designing many static partitions. Databricks recommends liquid clustering for new tables and describes it as replacing many traditional partitioning/Z-order use cases.

Example:

```
ALTER TABLE customer
CLUSTER BY (customer_id);
```

Solution 4 — Deduplicate source

If source has:

customer_id		update_time
101		10:00
101		10:01
101		10:02

don't MERGE all three.

Keep latest:

```
ROW_NUMBER() OVER (
  PARTITION BY customer_id
  ORDER BY update_time DESC
)
```

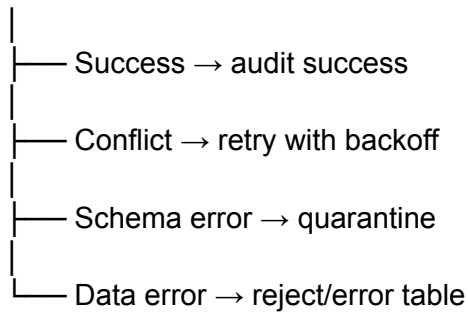
Solution 5 — Low-shuffle MERGE

Modern Databricks runtimes optimize MERGE so that unmodified rows are handled more efficiently, reducing shuffle.

Error handling

If MERGE fails:

MERGE



Don't blindly retry indefinitely.

Use:

Retry 1 → 30 sec

Retry 2 → 2 min

Retry 3 → 5 min

Then alert

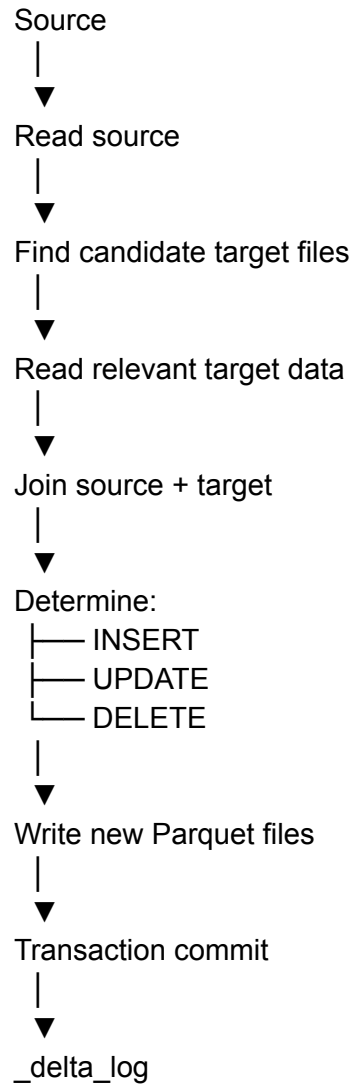
4. What actually happens internally during Delta MERGE?

This is where you demonstrate that you understand Delta rather than simply knowing SQL syntax.

Suppose:

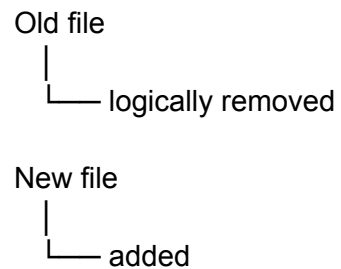
```
MERGE INTO target t
USING source s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

Conceptually:



Delta does not generally modify a Parquet file in place.

Instead:



The transaction log records the changes.

Example

Before:

file-A.parquet
file-B.parquet
file-C.parquet

Update one row in B.

Conceptually:

file-B.parquet
|
X removed

file-B-new.parquet
|
+ added

The table snapshot now references:

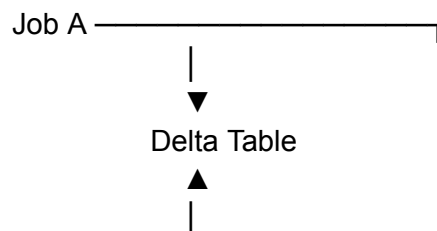
A
B-new
C

This is one reason Delta provides ACID-style transactional behavior.

5. Two jobs simultaneously update the same Delta table. What happens?

Delta uses transactional concurrency control.

Example:



Job B _____

Both jobs can read a snapshot.

At commit time, Delta checks whether the transaction conflicts with another transaction.

Possible result:

Job A → COMMIT SUCCESS

Job B → CONFLICT

↓

RETRY / FAIL

Current Databricks documentation describes conflict types such as [ConcurrentAppendException](#), [ConcurrentDeleteReadException](#), [ConcurrentDeleteDeleteException](#), [MetadataChangedException](#), and [ConcurrentTransactionException](#).

Example

Job A:

UPDATE customer WHERE id = 100

Job B:

UPDATE customer WHERE id = 100

Potential conflict.

But:

Job A:

UPDATE customer WHERE id = 100

Job B:

UPDATE customer WHERE id = 999999

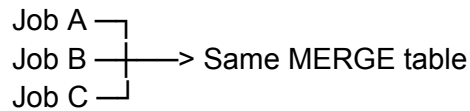
may be able to proceed depending on table layout and concurrency features.

Modern Delta configurations such as deletion vectors/liquid clustering can improve concurrency behavior.

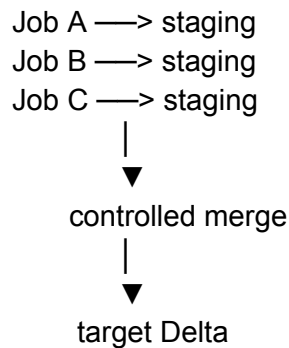
Production strategy

If the same table is frequently updated:

Bad architecture:



Prefer:



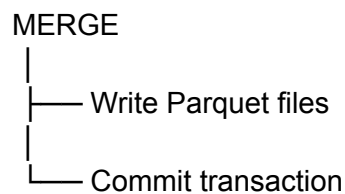
Or partition workloads by independent keys where possible.

6. Pipeline failed after writing files but before transaction completion. Are files visible?

Not as part of the Delta table snapshot.

This is an important distinction.

Suppose:



Failure occurs:

Write files ✓

Commit transaction ✗

The physical files may exist in storage, but they are not referenced by the committed Delta transaction log.

Therefore:

SELECT * FROM table

will not normally expose those uncommitted files.

Conceptually:

ADLS

```
data/
├── old-file.parquet    ← referenced
├── new-file.parquet    ← orphan/uncommitted
└── _delta_log/
    ├── 000001.json
    └── 000002.json
```

The table snapshot only sees files referenced by `_delta_log`.

Later cleanup processes can remove unused files according to retention policies.

Interview answer

“Delta separates physical file creation from transactional visibility. A file written before the transaction commits is not part of the committed Delta snapshot. Readers continue to see the previous consistent snapshot.”

This is one of the key advantages of Delta’s transaction model.

7. What is

`_delta_log`

and why is it important?

`_delta_log` is the **transaction log** of a Delta table.

Example:

```
customer/
|
|— part-0001.parquet
|— part-0002.parquet
|— part-0003.parquet
|
|— _delta_log/
    |— 000000.json
    |— 000001.json
    |— 000002.json
    |— ...
```

It records table changes.

Conceptually:

Version 0

↓

Version 1

↓

Version 2

↓

Version 3

Each transaction can record actions such as:

- add file
- remove file
- metadata change
- protocol change
- commit information

Why important?

It enables:

ACID transactions

Readers see consistent table versions.

Time travel

```
SELECT *  
FROM customer VERSION AS OF 100;
```

Rollback/recovery

You can inspect previous table versions.

Concurrent writes

Delta uses the transaction log to determine whether transactions conflict.

Streaming

Streaming readers/writers use Delta's transaction semantics for reliable processing.

Critical operational rule

Never treat `_delta_log` like ordinary data.

Do not:

manually delete `_delta_log` files

Cloud lifecycle policies must not accidentally remove the transaction log; doing so can make a Delta table inaccessible.

8. CDC pipeline receives duplicates and out-of-order events. How do you handle it?

This is a very common production problem.

Suppose:

Event 1:
customer=100
version=10

Event 2:
customer=100
version=12

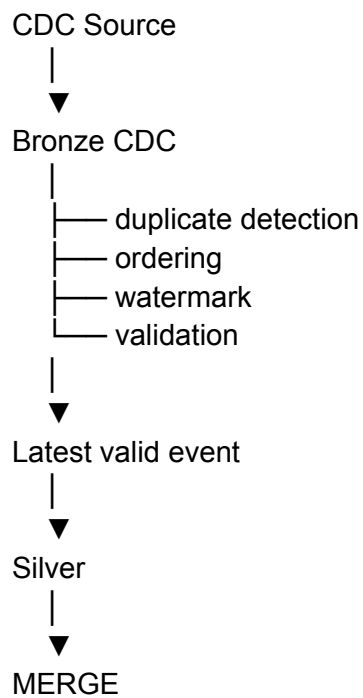
Event 3:
customer=100
version=11

Arrival order:

10 → 12 → 11

We should not blindly process arrival order.

Architecture



Deduplication

Use a unique event identifier:

event_id
transaction_id
LSN
SCN
source_sequence_number

For streaming workloads, Databricks supports watermark-based deduplication such as `dropDuplicatesWithinWatermark`.

Example:

```
df = (  
    df  
    .withWatermark("event_time", "2 hours")  
    .dropDuplicatesWithinWatermark(["event_id"])  
)
```

Out-of-order events

Use a sequence number.

For example:

customer	version	value
----- ----- -----		
100	10	A
100	12	C
100	11	B

Only apply version 12 as latest.

```
ROW_NUMBER() OVER (  
    PARTITION BY customer_id  
    ORDER BY source_version DESC  
)
```

Error handling

Late event:

event_time < watermark



Late-data/quarantine

Duplicate:

duplicate event_id



audit / ignore

Invalid sequence:

version <= current_version



ignore or quarantine

9. Compare Watermark vs CDC vs CDF

This is a question where candidates frequently confuse three completely different concepts.

Concept	Meaning
Watermark	Controls how much late event-time data streaming state waits for
CDC	Captures changes from a source system
CDF	Delta Change Data Feed exposes changes made to a Delta table

Watermark

Suppose:

Event time:

10:00

10:01

10:02

...

Watermark:

event_time - 10 minutes

means the engine can eventually consider data older than that threshold as too late.

It's primarily about **stream processing state and late data**.

It is not a database CDC mechanism.

CDC

CDC means:

INSERT

UPDATE

DELETE

coming from a source.

Example:

SQL Server



CDC / Debezium / native log



Kafka

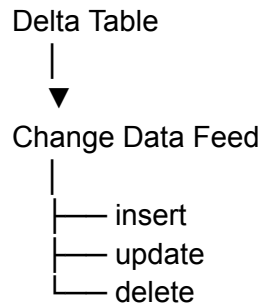


Databricks

CDF

Delta Change Data Feed lets downstream consumers read row-level changes from a Delta table.

Conceptually:



Easy interview explanation

“CDC is a change-capture architecture or source capability. CDF is a Delta feature for exposing changes from a Delta table. Watermark is a Structured Streaming mechanism for handling event-time progress and late-arriving data.”

10. CDC pipeline was down for 8 hours. How do you recover?

Do not simply restart from “now”.

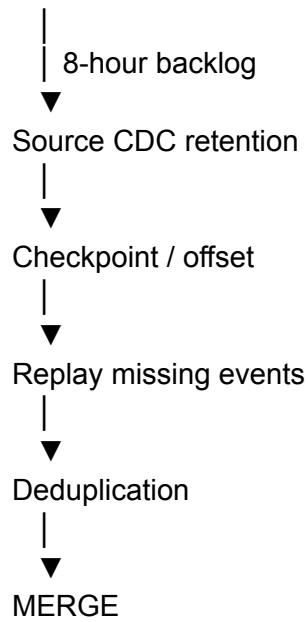
First determine the source’s recovery mechanism.

For example:

SQL Server CDC → LSN
Oracle → SCN
Kafka → offset
Event Hub → sequence/offset
Files → Auto Loader checkpoint

Recovery architecture

CDC Source



Step 1 — Find last successful checkpoint

Example:

Last processed LSN = 10000

Current source:

LSN = 15000

Need:

10001 → 15000

Step 2 — Verify source retention

If source only retains CDC for 4 hours:

Pipeline down = 8 hours

CDC retention = 4 hours

you cannot simply replay from the original LSN.

You need:

Backup

+
Full/incremental reconciliation

Step 3 — Replay

Process missing events.

Use idempotency:

event_id
LSN
transaction_id

so retries don't create duplicates.

Step 4 — Validate

Compare:

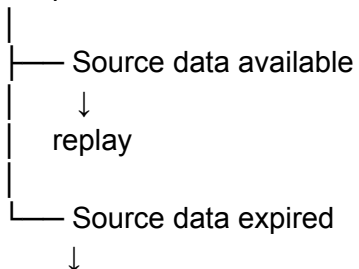
Source count
vs
Target count

and preferably:

checksum
max LSN
min LSN
record counts

Error scenario

Checkpoint valid



reconciliation



backfill

Strong interview answer

“I would recover from the last durable source offset/LSN rather than resetting the pipeline. First I verify CDC retention, then replay the missing range with idempotent processing. If the source retention has expired, I would perform a source-to-target reconciliation or targeted backfill.”

11. Spark job has 500 executors but only 20 are busy. What do you investigate?

This is a **Spark parallelism** question.

Don't immediately say:

“Add more executors.”

You already have 500.

First question

How many tasks are running?

Suppose:

Executors = 500

Tasks = 20

You cannot use 500 executors effectively with only 20 parallel tasks.

Investigate Spark UI

Look at:

Jobs



Stages



Tasks

Check:

- number of partitions
 - task duration
 - input size
 - shuffle size
 - skew
 - failed tasks
 - spill
 - executor utilization
-

Example

1 TB data



20 partitions



20 active tasks

500 executors are useless.

You need appropriate partitioning.

Possible causes

1. Too few partitions

`df.rdd.getNumPartitions()`

2. Data skew

Partition 1 → 500 GB

Partition 2 → 1 GB

Partition 3 → 1 GB

...

3. Single-threaded operation

Examples:

- global aggregation
- single partition
- `coalesce(1)`
- poorly designed UDF

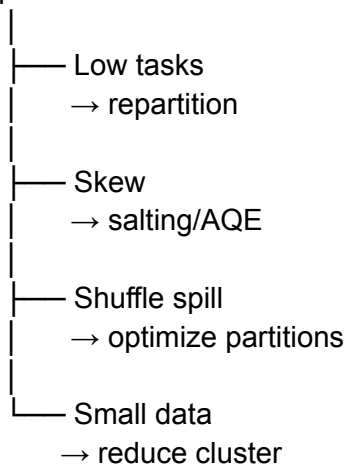
4. Driver bottleneck

5. Small input

If the job processes only 5 GB, 500 executors are unnecessary.

Error/performance solution

Spark UI



12. Explain a real data-skew problem and how you solve it

Suppose:

Orders table

contains:

customer_id

Most customers have 100 orders.

But:

customer_id = AMAZON

has:

500 million orders

Join

Orders

JOIN

Customers

Spark hashes:

customer_id

One partition receives enormous amounts of data.

Partition 1 → 500M records

Partition 2 → 1M

Partition 3 → 1M

...

Result:

499 tasks finish

1 task runs for 60 minutes

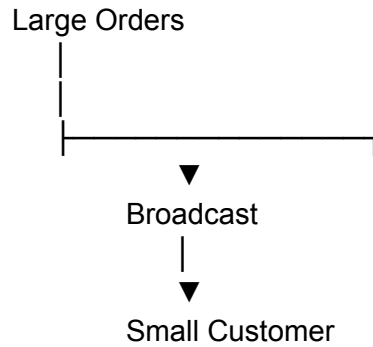
This is skew.

Solution 1 — AQE

Enable Adaptive Query Execution/skew handling where applicable.

Solution 2 — Broadcast

If the other table is genuinely small:



Solution 3 — Salting

Transform:

AMAZON

into:

AMAZON_1

AMAZON_2

AMAZON_3

...

AMAZON_100

Spread the hot key.

Solution 4 — Pre-aggregate

Instead of joining 500M rows:

500M orders



aggregate



10M rows

↓
join

How to identify it

Spark UI:

Task duration:

1 → 2 sec

2 → 2 sec

3 → 3 sec

...

20 → 45 min

That is a classic skew signature.

13. When can Broadcast Join make performance worse?

Broadcast joins are not automatically better.

Example:

```
broadcast(large_table)
```

Suppose:

Table = 50 GB

Executors = 500

You potentially create a huge broadcast burden.

Problems

1. Executor memory pressure

50 GB table

x

many executors

can create significant memory pressure.

2. GC

Executors spend time doing garbage collection.

3. OOM

Potential:

OutOfMemoryError

4. Network overhead

Broadcast must distribute the data.

5. Wrong statistics

Spark may think a table is small when it isn't.

When broadcast is appropriate

Fact = 10 TB

Dimension = 100 MB

Excellent candidate.

Fact = 10 TB

Dimension = 30 GB

Be careful.

Interview answer

“I use broadcast when the build side is genuinely small enough to fit comfortably in executor memory. I validate the physical plan, broadcast size, executor memory and GC behavior. I don't force broadcast merely because one table is smaller than the other.”

14. Spark job suddenly takes 3 hours instead of 30 minutes. No code changed. What do you check?

This is a **production troubleshooting** question.

I would use this sequence:

1. Input volume
↓
 2. File count
↓
 3. Data skew
↓
 4. Query plan
↓
 5. Shuffle
↓
 6. Cluster
↓
 7. Storage
↓
 8. Concurrent workloads
↓
 9. Runtime/configuration
-

1. Input volume

Yesterday:

100 GB

Today:

3 TB

Code didn't change.

2. Small files

Yesterday:

10,000 files

Today:

10 million files

File discovery overhead can explode.

3. Data skew

Maybe today's data has a hot key.

4. Spark UI

Check:

Input

Shuffle read

Shuffle write

Spill

Task duration

Failed tasks

5. Query plan

Run:

EXPLAIN FORMATTED

Look for:

BroadcastHashJoin

SortMergeJoin
Exchange
Filter
Scan

6. Cluster

Check:

executor count
executor memory
CPU utilization
autoscaling
driver health

7. Delta layout

Check:

file count
data skipping
statistics
clustering
small files

Delta data-skipping statistics can significantly reduce files/bytes read when filters align with available statistics.

8. Concurrent jobs

Maybe:

Job A
Job B
Job C
Job D

are competing for resources.

15. OPTIMIZE vs VACUUM vs Clustering

This is another important interview question.

Feature	Purpose
OPTIMIZE	Rewrite/compact files and improve physical layout
VACUUM	Remove old unused data files
Liquid clustering	Organize data according to clustering keys

OPTIMIZE

Suppose:

10 million tiny files

OPTIMIZE can compact them.

Before:

1 MB

2 MB

3 MB

1 MB

...

After:

512 MB

512 MB

512 MB

Databricks recommends frequent compaction where small files are a problem.

VACUUM

OPTIMIZE does not simply mean:

delete old files.

Old files remain because Delta may need them for:

- time travel
- concurrent readers
- recovery

VACUUM removes files that are no longer needed and have passed the retention threshold.

Liquid clustering

Modern Databricks recommendation:

Liquid clustering

instead of manually designing complex partitions/ZORDER for new tables.

Example:

```
ALTER TABLE sales  
CLUSTER BY (customer_id, event_date);
```

Liquid clustering allows the data layout to evolve without the rigid boundaries of traditional partitioning.

Easy analogy

OPTIMIZE

= organize the room

CLUSTERING

= organize things based on what you search for

VACUUM

= throw away things you no longer need

16. Why is over-partitioning dangerous?

Suppose:

1 TB table

partitioned by:

customer_id

and there are:

10 million customers

You can end up with huge numbers of tiny partitions/files.

customer=1/

customer=2/

customer=3/

...

customer=10M/

That's terrible.

Problems

1. Small files

10 MB

5 MB

2 MB

...

2. Metadata overhead

Spark must discover/manage huge numbers of files.

3. Slow queries

4. Slow writes

5. Difficult maintenance

6. Poor scalability

Databricks' current guidance is increasingly to avoid unnecessary partitioning and use liquid clustering for new Delta tables; custom partitioning should have a clear performance justification.

Good partition

For example:

year/month/day

if queries commonly filter by date.

But even that should be evaluated against current Databricks table-layout capabilities.

Bad partition

customer_id
transaction_id
timestamp

especially high-cardinality values.

17. Delta table is 30 TB and queries went from 5 seconds to 2 minutes. How investigate?

Do not immediately run:

OPTIMIZE

First investigate.

Step 1 — Check bytes scanned

Question:

30 TB table

How much data does query actually scan?

If:

Query scans 25 TB

then 2 minutes may be reasonable.

If:

Query scans 5 GB

then something is wrong.

Step 2 — Data skipping

Check whether filters allow Delta to skip files.

Example:

WHERE customer_id = 100

If statistics are poor, many files may be read.

Delta maintains file-level statistics such as min/max values and record counts for data skipping.

Step 3 — File count

Before:

100,000 files

After:
50 million files

Small-file explosion can hurt badly.

Step 4 — Data layout

Check:

partitioning
clustering
ZORDER for legacy layouts
statistics

For modern tables, investigate liquid clustering.

Step 5 — Query plan

Look for:

full table scan
large shuffle
sort merge join
broadcast failure
Cartesian join

Step 6 — Statistics

Run appropriate statistics maintenance where necessary.

For example, Databricks supports recomputing Delta statistics with:

```
ANALYZE TABLE table_name  
COMPUTE DELTA STATISTICS;
```

in supported runtimes.

Step 7 — Concurrency

Maybe the table is being heavily written while queries execute.

Databricks query performance insights explicitly surfaces concurrent writes as a possible contributor.

Investigation flow

Query slower

- More data?
 - More files?
 - Data skipping degraded?
 - Bad statistics?
 - Layout changed?
 - Query plan changed?
 - Shuffle increased?
 - Concurrent workload?
-

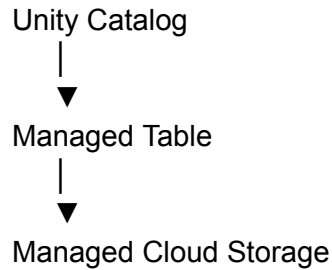
18. Managed tables vs External tables in Unity Catalog

This is important for Unity Catalog interviews.

Managed table

Unity Catalog controls the table's storage location and lifecycle.

Conceptually:



You don't manually manage the physical table location in the normal workflow.

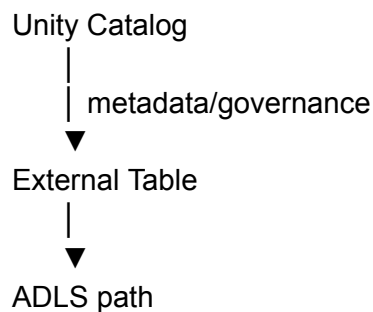
Databricks currently recommends managed tables where possible because they simplify governance and lifecycle management.

External table

You specify the storage location.

```
CREATE TABLE sales
USING DELTA
LOCATION 'abfss://...';
```

Conceptually:



The data lifecycle remains under the customer's/external system's control.

When use managed?

Use managed when:

- Databricks owns the data lifecycle
- you want simplified governance
- you want automatic optimization features

- the data is primarily consumed through Databricks
-

When use external?

Use external when:

- data already exists in ADLS
 - another platform needs direct access
 - you must retain a specific storage path
 - the storage lifecycle is externally managed
-

Interview answer

“Managed means Unity Catalog manages the storage location and lifecycle of the table. External means Unity Catalog governs the metadata and access, but the table points to a customer-managed storage path whose data lifecycle remains external.”

19. Storage Credential vs External Location

This is frequently asked in Unity Catalog interviews.

Think:

Storage Credential

+

|

▼

External Location

|

▼

ADLS Path

Storage Credential

A storage credential represents **how Databricks authenticates to cloud storage**.

For Azure, think of the relevant identity/authentication mechanism.

Conceptually:

Storage Credential

=

"Who are you when accessing storage?"

External Location

External location combines:

Storage path

+

Storage credential

For example:

External Location

Name: prod_sales

Path: abfss://sales@storage...

Credential: prod_identity

Databricks defines an external location as an object that combines a cloud-storage path with a storage credential.

Analogy

Imagine a building.

Storage Credential

= employee ID

External Location

= employee ID + building/floor

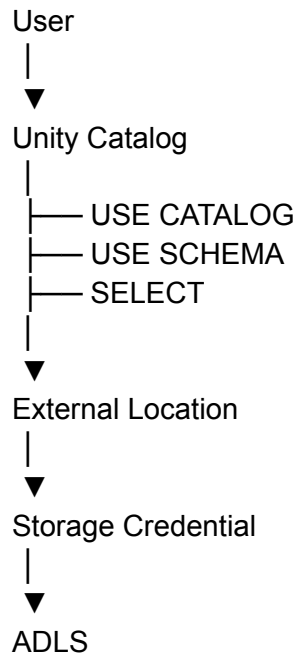
Credential answers:

Who can authenticate?

External location answers:

Where can that identity access?

Security model



Don't give every user direct ADLS permissions when Unity Catalog should govern access. Databricks explicitly recommends using Unity Catalog governance rather than bypassing it with direct cloud-storage access.

20. Design an end-to-end production CDC architecture

This is the **most important architecture question** of the 20.

Let's design one.

Production CDC architecture

Assume:

SQL Server / SAP / Oracle

is the source.

We want:

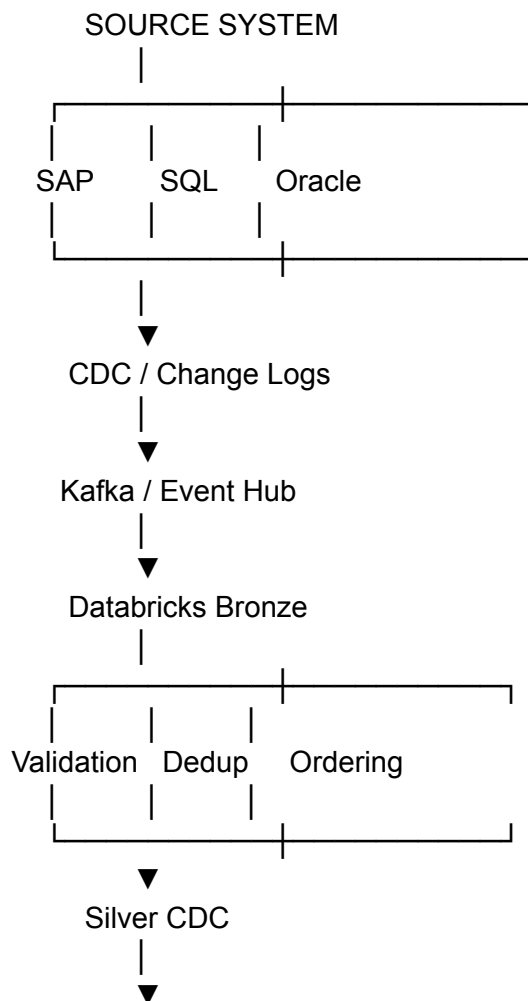
Source changes

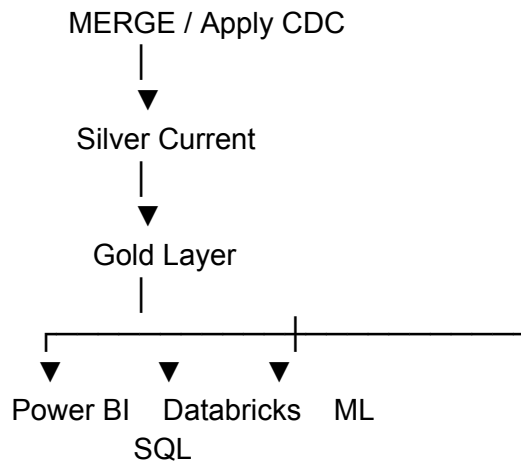
→ Databricks

→ Delta

→ BI

Architecture





Bronze CDC table

Keep the raw CDC events.

Example:

event_id
source_system
table_name
operation
customer_id
event_time
source_lsn
before_value
after_value
ingestion_time

Example:

event_id	operation	customer_id	LSN
1001	INSERT	101	5001
1002	UPDATE	102	5002
1003	DELETE	103	5003

Silver CDC

Perform:

Schema validation



Deduplication



Ordering



Late event handling



Data quality

Then:

INSERT

UPDATE

DELETE

are applied to the current-state table.

MERGE pattern

Conceptually:

```
MERGE INTO customer t
USING latest_customer_changes s
ON t.customer_id = s.customer_id
```

```
WHEN MATCHED
AND s.operation = 'DELETE'
THEN DELETE
```

```
WHEN MATCHED
AND s.operation = 'UPDATE'
THEN UPDATE SET *
```

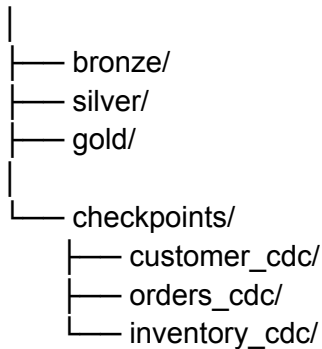
```
WHEN NOT MATCHED
AND s.operation = 'INSERT'
THEN INSERT *;
```

In production, I'd usually preprocess the source to **one authoritative/latest event per business key per batch/window** before MERGE.

Checkpointing

Every streaming pipeline needs durable checkpoint state.

ADLS



Don't share checkpoints between independent streams.

Databricks notes that starting two streaming queries with the same checkpoint concurrently can cause `ConcurrentTransactionException`; changing a checkpoint location also effectively starts a new stream.

CDC error handling

A production CDC architecture should explicitly handle these scenarios:

1. Duplicate event

event_id already processed



ignore

2. Out-of-order event

LSN 105

LSN 107

LSN 106

Use source sequence/LSN and ordering rules.

3. Late event

event older than watermark



late-event table

or process it through a controlled reconciliation mechanism.

4. Schema change

Example:

Source:

customer_id
name

New:

customer_id
name
email

Use controlled schema evolution rather than allowing arbitrary changes into production.

5. Poison message

One bad record shouldn't necessarily stop the entire pipeline.

Kafka



Databricks



Valid → Silver

|
└─ Invalid → Quarantine

Quarantine table:

cdc_error_records

event_id
source
payload
error_code
error_message
ingestion_time
pipeline_run_id

6. Pipeline failure

Use:

checkpoint

+

source offset

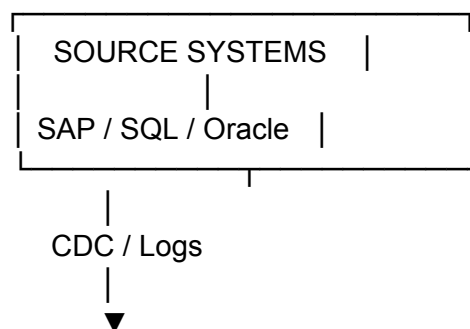
+

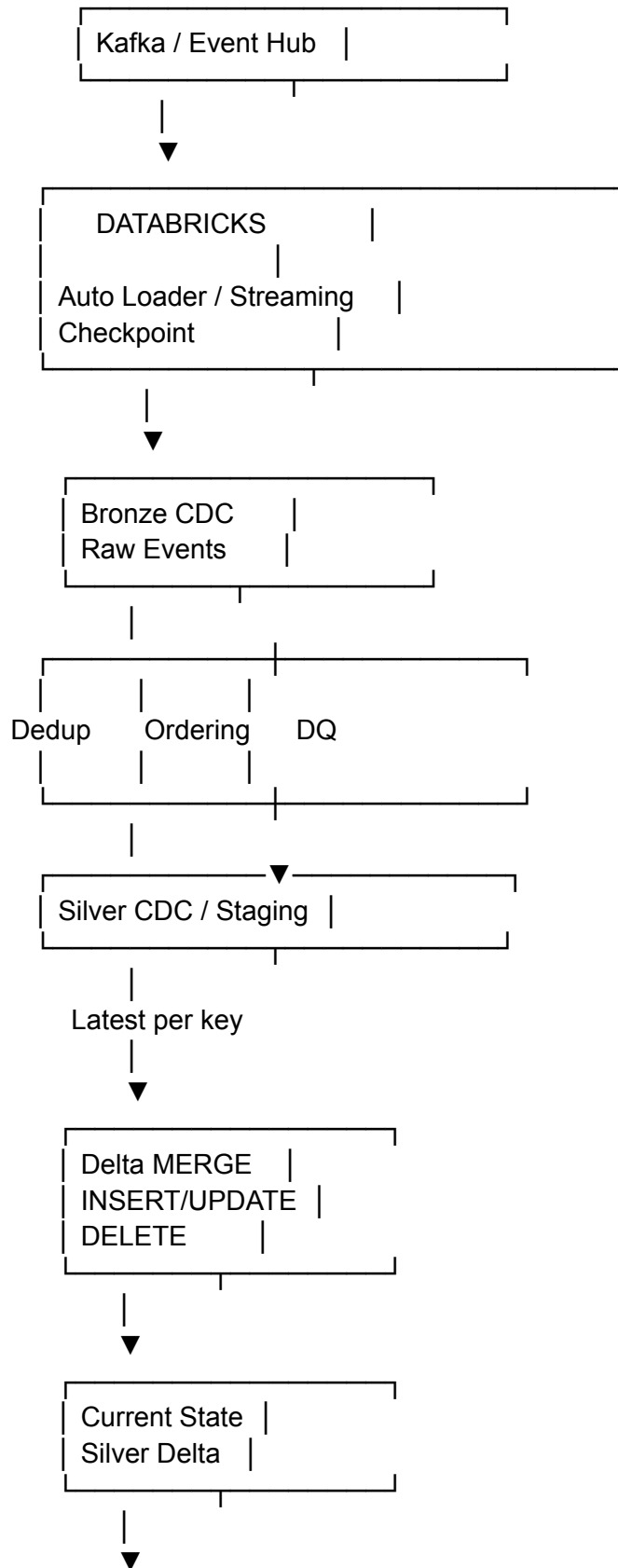
idempotent processing

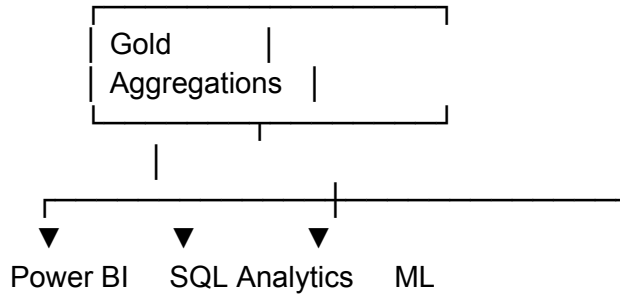
so the pipeline can resume.

Complete production CDC architecture

Putting everything together:







Production Error Handling Framework

For an interview, don't just say **"we retry the job."**

Explain a proper framework.

1. Data-level errors

Invalid schema
Invalid datatype
Null mandatory field
Invalid business key
Duplicate event

→ quarantine table.

2. Infrastructure errors

Cluster failure
Storage timeout
Network failure
Executor failure

→ automatic retry.

3. Transaction errors

ConcurrentAppendException
ConcurrentDeleteReadException

MetadataChangedException

→ retry with exponential backoff, or redesign concurrent workloads where conflicts are frequent.

4. Streaming errors

Checkpoint corruption

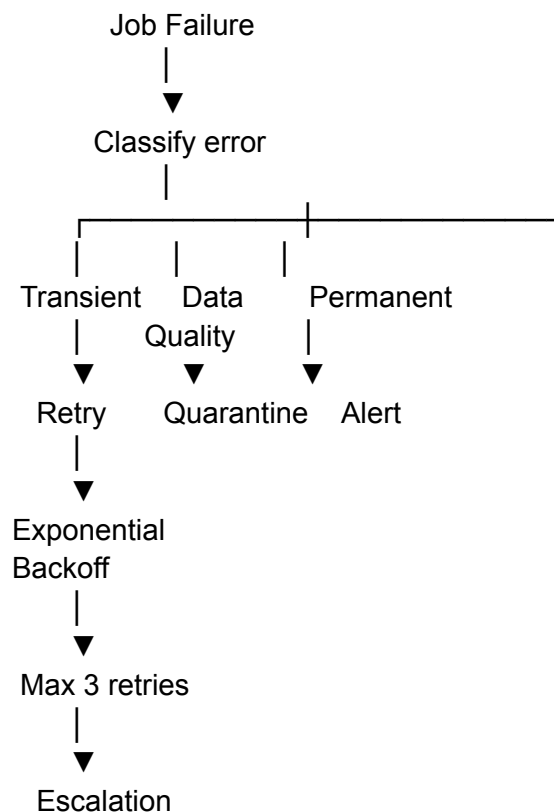
Source unavailable

Offset expired

Schema change

→ recover from durable checkpoint/source offset or perform controlled backfill.

A good production retry framework



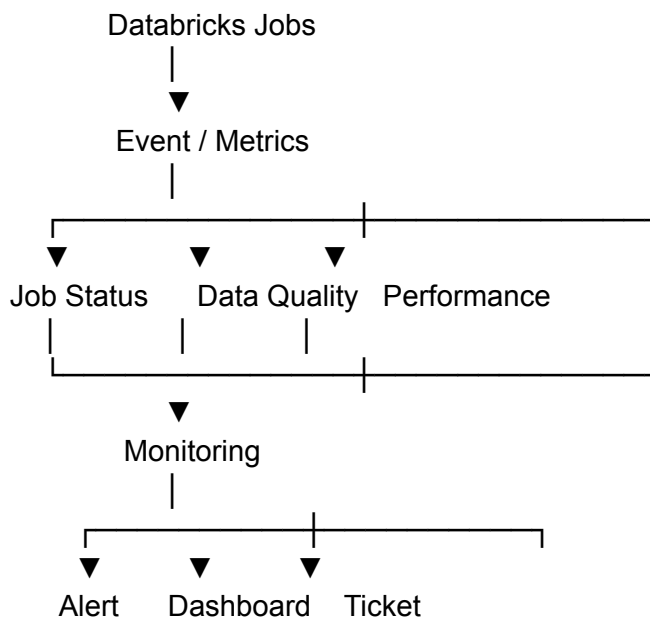
Example:

Retry 1 → 30 seconds

Retry 2 → 2 minutes
Retry 3 → 5 minutes
Failure → Pager/Teams/Email

Monitoring Architecture

For a real production system, I would add:



Monitor at least:

Pipeline

Success/failure
Duration
Retry count
Last successful run

Data

Records read
Records written
Records rejected
Duplicate count
Late records

Spark

CPU
Memory
Shuffle
Spill
Task skew
Executor utilization

Delta

File count
Table size
OPTIMIZE history
MERGE duration
Concurrent writes

CDC

Current LSN/offset
Source LSN
Lag
Events/sec
Duplicate events
Late events

20 Questions — What an interviewer is really testing

#	Question	What they are testing
1	5 TB/day architecture	Architecture & scalability
2	Auto Loader	Ingestion architecture
3	2 TB MERGE	Delta performance

4	Internal MERGE	Delta internals
5	Concurrent writes	ACID/concurrency
6	Files before commit	Transaction semantics
7	<code>_delta_log</code>	Delta internals
8	Duplicate/out-of-order CDC	Streaming/CDC
9	Watermark/CDC/CDF	Conceptual knowledge
10	8-hour CDC outage	Recovery
11	500 executors	Spark parallelism
12	Data skew	Spark optimization
13	Broadcast	Join optimization
14	Sudden slowdown	Troubleshooting
15	OPTIMIZE/VACUUM	Delta maintenance

16	Over-partitioning	Data layout
17	30 TB slow queries	Performance engineering
18	Managed/external	Unity Catalog
19	Credential/location	Cloud governance
20	End-to-end CDC	Senior architecture

The most important interview pattern

For almost every senior Databricks question, answer using this sequence:

1. Identify the problem

What exactly is slow/failing?

2. Explain the root cause

Why is it happening?

3. Give the architecture/design

How would you build it?

4. Give the optimization

How would you make it scalable?

5. Give failure handling

What happens if it fails?

6. Give monitoring

How will you know it's healthy?

For example, don't answer Q3 merely with:

"Use partition pruning."

A senior-level answer is:

"First I'd verify whether the MERGE is scanning the entire target. Since only 20 GB changes out of 2 TB, I'd make the source incremental, deduplicate it by business key, constrain candidate target files using the merge predicate and data layout, and use Delta statistics/data skipping or liquid clustering. I'd inspect the Spark UI to verify bytes scanned and shuffle. I'd also handle concurrent-write conflicts with bounded retries and monitor MERGE duration, files scanned, and records affected."

That style demonstrates **architecture + Spark + Delta + production operations**, rather than just memorized commands.

The 5 areas I would prioritize for your interview

Given these questions, I'd spend the most preparation time on:

1. **Delta Lake internals** — `_delta_log`, snapshots, MERGE, ACID, concurrency
2. **Spark performance** — partitions, shuffle, skew, AQE, broadcast
3. **CDC** — LSN/offset, deduplication, ordering, watermark, CDF
4. **Production architecture** — Bronze/Silver/Gold, Auto Loader, checkpoints, recovery
5. **Unity Catalog** — managed/external tables, storage credentials, external locations

These five areas cover most of the deeper follow-up questions an interviewer can derive from the 20 questions.